

Databases and Applications

Unit III

Normalization: 1NF, 2NF, Functional Dependencies

Pooja T S

Computer Applications



Databases and Applications

Normalization – Introduction

- ▶ Normalization is a process that improves database design by generating relations of higher normal forms.
 - Normal forms discussed: 1NF, 2NF, 3NF, BCNF
 - Objective: ensure every dependency is on the key, the whole key, and nothing but the key
 - Reduces redundancy and simplifies update semantics



Databases and Applications

Hierarchy of Normal Forms

- ▶ Normal forms follow a sequence:
 - 1NF is weakest
 - 2NF stronger than 1NF
 - 3NF stronger than 2NF
 - BCNF strongest
- ▶ Any relation in BCNF is also in 3NF, in 2NF, and in 1NF.



Databases and Applications

Why Normalization Matters

- ▶ Higher normal forms reduce redundancy and prevent integrity problems.
- ▶ Benefits include:
 - Simpler update semantics
 - Lower redundancy
 - More consistent relational structures



Databases and Applications

Functional Dependencies – Definition

- ▶ A functional dependency $A \rightarrow B$ means:
 - If two records have the same A-value, they must have the same B-value
 - A uniquely determines B
- ▶ Example:
 - $\text{EmpNum} \rightarrow \text{EmpEmail}$



- Functional Dependencies from the employee relation:

EmpNum	EmpEmail	EmpFname	EmpLname
123	jdoe@abc.com	John	Doe
456	psmith@abc.com	Peter	Smith
555	alee1@abc.com	Alan	Lee
633	pdoe@abc.com	Peter	Doe
787	alee2@abc.com	Alan	Lee



- ▶ If EmpNum is the primary key, then:
 - $\text{EmpNum} \rightarrow \text{EmpEmail}$
 - $\text{EmpNum} \rightarrow \text{EmpFname}$
 - $\text{EmpNum} \rightarrow \text{EmpLname}$
- ▶ Each employee has one email, firstname, lastname determined by EmpNum.



Databases and Applications

Determinants

- ▶ In the FD $\text{EmpNum} \rightarrow \text{EmpEmail}$:
 - EmpNum is the **determinant**
 - Determinant = attribute on the LHS of an FD



Databases and Applications

Types of Dependencies

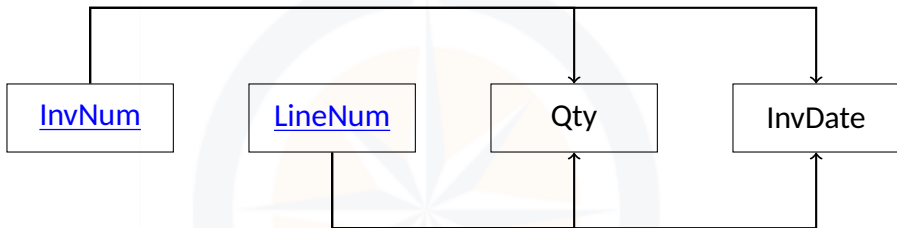
- ▶ Normalization uses:
 - **Partial dependency** – attribute depends on part of a composite key
 - **Transitive dependency** – $A \rightarrow B$ and $B \rightarrow C$ implies $A \rightarrow C$
- ▶ These violations determine which normal form is achieved.



Databases and Applications

Partial Dependency – Definition

- ▶ A partial dependency exists when:
 - B depends on A
 - A is part of a composite candidate key
- ▶ Only occurs when candidate keys have multiple attributes.



- ▶ Candidate key: (InvNum, LineNum)
- ▶ InvDate depends only on InvNum.
- ▶ Therefore:
 - InvDate is partially dependent on (InvNum, LineNum)
 - Violates 2NF



1NF – Definition

- ▶ A relation is in 1NF if:
 - All attribute values are single-valued and atomic
 - No repeating groups
 - No multivalued attributes



Databases and Applications

1NF – Example of Violation

- The following relation is **not in 1NF** due to multivalued attributes:

EmpNum	EmpPhone	EmpDegrees
123	233-9876	–
333	233-1231	BA, BSc, PhD
679	233-1231	BSc, MSc

- Issues:
- EmpDegrees has multiple values for a single employee
 - Violates atomicity and 1NF requirements



Databases and Applications

1NF – Example of Violation

- ▶ Example of not being in 1NF:
 - EmpDegrees = BA, BSc, PhD (multi-valued)
 - Employee 679: BSc, MSc
- ▶ Multi-valued fields violate atomicity.



Databases and Applications

Converting to 1NF

► Solution:

- Create separate rows for each degree
- Split into two relations:
 - Employee(EmpNum, EmpPhone)
 - EmployeeDegree(EmpNum, EmpDegree)



Databases and Applications

Converting to 1NF

- ▶ To achieve 1NF, multi-valued attributes must be separated into atomic values.
- ▶ Step 1: Create a base **Employee** relation with atomic attributes.

Employee Table

EmpNum	EmpPhone
123	233-9876
333	233-1231
679	233-1231

- ▶ This removes the multi-valued EmpDegrees attribute from the main relation.



Databases and Applications

Converting to 1NF

- ▶ Step 2: Create a separate **EmployeeDegree** relation to store all degree values atomically.

EmployeeDegree Table

EmpNum	EmpDegree
333	BA
333	BSc
333	PhD
679	BSc
679	MSc

- ▶ These two tables, when joined, reconstruct the original data without violating 1NF.



Databases and Applications

2NF – Definition

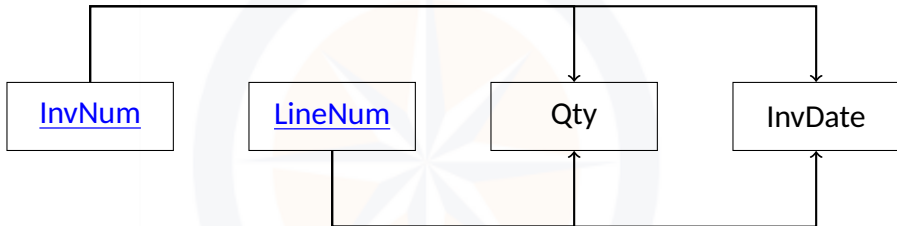
- ▶ A relation is in 2NF if:
 - It is already in 1NF
 - Every non-key attribute is fully dependent on each candidate key
 - No partial dependencies exist



Databases and Applications

2NF – Example Analysis

- Consider the relation shown below:



- Issues:

- Partial dependency: InvDate depends only on InvNum
- Composite key attributes InvNum and LineNum
- Repeated invoice dates cause redundancy
- Violates 2NF



Databases and Applications

2NF – Example Analysis

- ▶ Relation: InvLine(InvNum, LineNum, ProdNum, Qty, InvDate)
- ▶ Issues:
 - InvDate depends only on InvNum → partial dependency
 - Repeated InvDate across invoice lines → redundancy



Databases and Applications

2NF – Decomposition

- ▶ Decompose into two relations:
 - InvLine(InvNum, LineNum, ProdNum, Qty)
 - Invoice(InvNum, InvDate)
- ▶ Removes partial dependency and redundancy.



Databases and Applications

2NF – Decomposition

We can *improve* the database by decomposing the relation into two relations:

Relation 1

<u>InvNum</u>	<u>LineNum</u>	ProdNum	Qty
---------------	----------------	---------	-----

Relation 2

<u>InvNum</u>	InvDate
---------------	---------

Databases and Applications

Unit III

3NF, Transitive Dependencies, BCNF

Pooja T S

Computer Applications



Transitive Dependency – Definition

- ▶ **Transitive Dependency**
- ▶ A transitive dependency exists in a relation R if:
 - We have functional dependencies $X \rightarrow Y$ and $Y \rightarrow Z$
 - X is a key (or contains a key) of the relation
 - Z is a **non-key** attribute
- ▶ Then $X \rightarrow Z$ holds *indirectly* via Y, and we say Z is **transitively dependent** on X.
- ▶ 3NF aims to **remove** such transitive dependencies of non-key attributes on a candidate key.



Transitive Dependency – Employee Example

- ▶ Consider the **Employee** relation from the 3NF discussion:

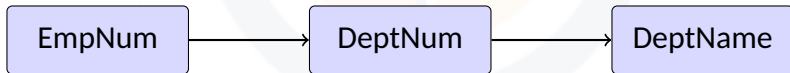
EmpNum	EmpName	DeptNum	DeptName
--------	---------	---------	----------

- ▶ Functional Dependencies:
 - $\text{EmpNum} \rightarrow \text{EmpName}$
 - $\text{EmpNum} \rightarrow \text{DeptNum}$
 - $\text{DeptNum} \rightarrow \text{DeptName}$
- ▶ The first two are direct dependencies from the candidate key EmpNum.



Transitive Dependency – Employee Example

- ▶ From the previous slide:
 - $\text{EmpNum} \rightarrow \text{DeptNum}$
 - $\text{DeptNum} \rightarrow \text{DeptName}$
- ▶ Therefore:
 - $\text{EmpNum} \rightarrow \text{DeptName}$ (indirect)
- ▶ This is a **transitive dependency** because DeptName is a non-key attribute depending on EmpNum *through* DeptNum.



- ▶ Because of this transitive dependency, the original Employee relation is **not in 3NF**.



Third Normal Form

- ▶ A relation is in **3NF** if the relation is in **1NF** and all determinants of non-key attributes are candidate keys.
 - That is, for any functional dependency: $X \rightarrow Y$, where Y is a non-key attribute (or a set of non-key attributes), X is a candidate key.
- ▶ This definition of 3NF differs from BCNF only in the specification of non-key attributes – 3NF is **weaker** than BCNF.
 - BCNF requires *all* determinants to be candidate keys.
- ▶ A relation in 3NF will not have any **transitive dependencies** of a non-key attribute on a candidate key through another non-key attribute.



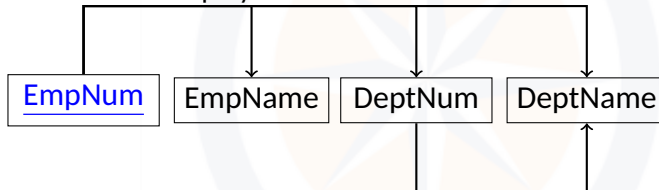
- ▶ Consider this **Employee** relation:

EmpNum	EmpName	DeptNum	DeptName
--------	---------	---------	----------

- ▶ EmpName, DeptNum, and DeptName are **non-key attributes**.
- ▶ DeptNum determines DeptName, a non-key attribute, and DeptNum is not a candidate key.
- ▶ From the slide:
 - Is the relation in 3NF? ... **no**
 - Is the relation in BCNF? ... **no**
 - Is the relation in 2NF? ... **yes**
 - Candidate keys are? ... **EmpNum**



- ▶ Consider the Employee relation:





► Functional dependencies:

- $\text{EmpNum} \rightarrow \text{EmpName}$
- $\text{EmpNum} \rightarrow \text{DeptNum}$
- $\text{DeptNum} \rightarrow \text{DeptName}$

► This creates a **transitive dependency**:

$\text{EmpNum} \rightarrow \text{DeptNum} \rightarrow \text{DeptName}$

- This violates 3NF because a non-key attribute depends on another non-key attribute.



- ▶ We correct the transitive dependency by decomposing the relation into 3NF relations:

- ▶ Relation 1

EmpNum	EmpName	DeptNum
--------	---------	---------

- ▶ Relation 2

DeptNum	DeptName
---------	----------

- ▶ Both resulting relations satisfy 3NF.



Databases and Applications

3NF but NOT BCNF

- ▶ Example: *Instructor teaches one course only; student takes a course and has one instructor.*

student_no	course_no	instr_no
------------	-----------	----------

- ▶ Functional dependencies:
 - student_no, course_no $\rightarrow$ instr_no
 - instr_no $\rightarrow$ course_no
- ▶ Since instr_no is not a candidate key, the relation is NOT in BCNF.



Databases and Applications

Boyce-Codd Normal Form (BCNF)

► A relation is in **BCNF** if:

- It is already in 3NF
- For every functional dependency $X \rightarrow Y$:
 - The determinant X is a **superkey (SK)**

► **Why BCNF?**

- 3NF still allows some anomalies.
- Non-superkey determinants cause redundancy.
- BCNF ensures all dependencies arise from keys only.

► **Characteristics**

- Every FD satisfies: LHS is a superkey.
- BCNF is stricter than 3NF.
- Removes all update, insertion, and deletion anomalies.

► **Observation**

- If a relation has only one candidate key, 3NF = BCNF.



Databases and Applications

Boyce-Codd Normal Form (BCNF)

► Common BCNF Violation Example

- Relation: *Teach*(*Course*, *Instructor*, *Room*)
- FDs:

$Course \rightarrow Instructor, \quad Instructor \rightarrow Room$

- Problem:
 - *Instructor* is not a superkey
- ⇒ Relation is in 3NF but **not in BCNF**.

► BCNF Decomposition Rule

- If $X \rightarrow Y$ violates BCNF (i.e., X not SK), decompose into:
 - $R_1 = X \cup Y$
 - $R_2 = R - (Y - X)$
- Decomposition is always **lossless**.
- Further decomposition may be needed until all relations satisfy BCNF.



Databases and Applications

BCNF – Decomposition

- ▶ We decompose the relation into BCNF relations:

student_no	instr_no
------------	----------

course_no	instr_no
-----------	----------

- ▶ The decomposition results in BCNF relations.



Databases and Applications

Normalisation – Visual Summary

1NF – Atomic values

2NF – No partial dependencies

3NF – No transitive dependencies

BCNF – LHS of every FD is a SK



Databases and Applications

Summary of Normalisation

- ▶ Normalisation improves database design by reducing redundancy and eliminating anomalies.
- ▶ **1NF – First Normal Form**
 - No repeating groups or multivalued attributes
 - All attribute values are atomic
- ▶ **2NF – Second Normal Form**
 - Must be in 1NF
 - No partial dependency on a composite key
 - Every non-key attribute depends on the *whole* key



Databases and Applications

Summary of Normalisation

► 3NF – Third Normal Form

- Must be in 2NF
- No transitive dependencies on key attributes
- All determinants must be candidate keys, except when the dependent attribute is a prime attribute

► BCNF – Boyce-Codd Normal Form

- Must be in 3NF
- For every functional dependency $X \rightarrow Y$, X must be a superkey
- Stronger than 3NF; removes remaining anomalies



Databases and Applications

Normalisation – Practice Question 1

► Consider a relation

$$R = (A, B, C, D, E)$$

with functional dependencies:

$$\{ CE \rightarrow D, D \rightarrow B, C \rightarrow A \}$$

► Tasks:

- Find all candidate keys of R .
- Determine the highest normal form satisfied by R (1NF, 2NF, 3NF).
- If the relation is not in 3NF, decompose it until it becomes 3NF.

At each decomposition step, form a new relation, recompute candidate keys, and identify its normal form.



► Given:

Relation: $R(A, B, C, D, E)$

FDs: $\{CE \rightarrow D, D \rightarrow B, C \rightarrow A\}$

► Candidate Keys

- Attributes that never appear on RHS: C, E
- Check $(CE)^+ = \{A, B, C, D, E\} \Rightarrow CE$ is a key.
- No proper subset of CE is a key $\Rightarrow$ **only candidate key: CE .**

► Highest Normal Form of R

- Partial dependency: $C \rightarrow A$ (non-prime A depends on part of key C).
- Transitive dependency: $CE \rightarrow D \rightarrow B$.
- Hence R violates 2NF and 3NF.
 $\Rightarrow$ **Best normal form of R : 1NF.**



► 3NF Decomposition

- From minimal cover, create relations:

$R_1(C, E, D)$ from $CE \rightarrow D$

$R_2(D, B)$ from $D \rightarrow B$

$R_3(C, A)$ from $C \rightarrow A$

- R_1 contains key CE for the original schema.
- In each relation, the determinant is a key $\Rightarrow$ all R_1, R_2, R_3 are in 3NF.



Databases and Applications

Normalisation – Practice Question 2

► Consider a relation

$$R = (A, B, C, D, E, F, G, H, I, J)$$

with the following functional dependencies:

$$\{ AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ \}$$

► Tasks:

- Determine all candidate keys of R .
- Identify the highest normal form satisfied by R (1NF, 2NF, 3NF).
- If the relation is not in 3NF, decompose it step-by-step until it reaches 3NF.

During each step, form a new relation, recompute its candidate keys, and check the normal form it satisfies.



► Given:

Relation: $R(A, B, C, D, E, F, G, H, I, J)$

FDs:

$$\{ AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ \}$$

► Candidate Keys

- Attributes never on RHS: $A, B \Rightarrow$ must appear in every key.
- Compute closure:

$$(AB)^+ = \{A, B, C, D, E, F, G, H, I, J\}$$

- Neither A^+ nor B^+ gives all attributes.
 $\Rightarrow$ **Only candidate key: AB.**



► Highest Normal Form

- Partial dependencies:

$A \rightarrow DE$ (only part of key)

$B \rightarrow F$ (only part of key)

- Transitive dependency:

$AB \rightarrow D \rightarrow IJ$

⇒ **R violates 2NF and 3NF.**

⇒ **Best normal form of R = 1NF.**



► 3NF Decomposition

- Using minimal cover, generate relations:

$R_1(A, B, C)$ from $AB \rightarrow C$

$R_2(A, D, E)$ from $A \rightarrow DE$

$R_3(B, F)$ from $B \rightarrow F$

$R_4(F, G, H)$ from $F \rightarrow GH$

$R_5(D, I, J)$ from $D \rightarrow IJ$

- R_1 contains the candidate key AB of the original relation.
- In every R_i , the determinant is a key $\Rightarrow$ each relation is in **3NF**.

Final decomposition:

$$\{R_1, R_2, R_3, R_4, R_5\}$$

Databases and Applications

Unit III

Document Models: JSON, BSON, Document DB Basics

Pooja T S

Computer Applications



► What is a Document Model?

- Stores data as documents instead of tables
- Each document groups related fields together
- Represents real-world entities in a natural way

► Why is it used?

- Extremely flexible and schema-less structure
- Ideal for nested, hierarchical data
- Widely used in modern web and mobile applications



► What is JSON?

- JavaScript Object Notation (lightweight format)
- Stores data as key-value pairs
- Human-readable and easy to exchange between systems

► JSON Structure

- Objects enclosed in { }
- Arrays enclosed in []
- Supports strings, numbers, booleans, arrays, and nested objects



Databases and Applications

JSON – Example

► Simple JSON Document

```
{  
  "name": "Aarathi",  
  "age": 23,  
  "skills": ["Python", "SQL"]  
}
```

► Why JSON is useful

- Clean way to represent a single entity
- Handles lists and nested fields easily
- Used in REST APIs, NoSQL databases, and configuration files



► What is BSON?

- Binary JSON format used internally by MongoDB
- Designed for fast reading and writing
- Supports additional types like Date, ObjectId, Binary

► Why not JSON alone?

- JSON lacks complex data types
- JSON is text-based → slower to parse
- BSON provides type metadata and indexing advantages



JSON vs BSON & Document Databases

► JSON vs BSON

- JSON → human-readable, lightweight, text format
- BSON → machine-readable binary, larger but faster
- BSON enables efficient database queries and indexes

► Document Databases

- Store information as JSON/BSON documents
- Schema-less: structure may vary between documents
- Examples: MongoDB, CouchDB, Firebase Firestore

Databases and Applications

Unit III

Document Models: JSON, BSON, Document DB Basics

Pooja T S

Computer Applications



► What is a Document Model?

- Stores data as documents instead of tables
- Each document groups related fields together
- Represents real-world entities in a natural way

► Why is it used?

- Extremely flexible and schema-less structure
- Ideal for nested, hierarchical data
- Widely used in modern web and mobile applications



► What is JSON?

- JavaScript Object Notation (lightweight format)
- Stores data as key-value pairs
- Human-readable and easy to exchange between systems

► JSON Structure

- Objects enclosed in { }
- Arrays enclosed in []
- Supports strings, numbers, booleans, arrays, and nested objects



Databases and Applications

JSON – Example

► Simple JSON Document

```
{  
  "name": "Aarathi",  
  "age": 23,  
  "skills": ["Python", "SQL"]  
}
```

► Why JSON is useful

- Clean way to represent a single entity
- Handles lists and nested fields easily
- Used in REST APIs, NoSQL databases, and configuration files



► What is BSON?

- Binary JSON format used internally by MongoDB
- Designed for fast reading and writing
- Supports additional types like Date, ObjectId, Binary

► Why not JSON alone?

- JSON lacks complex data types
- JSON is text-based → slower to parse
- BSON provides type metadata and indexing advantages



JSON vs BSON & Document Databases

► JSON vs BSON

- JSON → human-readable, lightweight, text format
- BSON → machine-readable binary, larger but faster
- BSON enables efficient database queries and indexes

► Document Databases

- Store information as JSON/BSON documents
- Schema-less: structure may vary between documents
- Examples: MongoDB, CouchDB, Firebase Firestore

Databases and Applications

Unit III

BASE Principles, CAP Theorem, RDBMS vs NoSQL

Pooja T S

Computer Applications



BASE Principles – NoSQL Reliability Model

► Why BASE emerged ?

- Traditional ACID models become restrictive in distributed clusters
- Large-scale web systems need availability even during partial failures
- BASE trades strict correctness for operational resilience and performance

► Detailed BASE Semantics

- **Basically Available:** System continues operations even if some nodes are unreachable
- **Soft-State:** Replicas may temporarily hold different versions of data
- **Eventual Consistency:** Background reconciliation ensures replicas converge



Databases and Applications

BASE Principles – NoSQL Foundations



► What is BASE?

- **Basically Available** – system remains available even during failures
- **Soft-state** – data may be temporarily inconsistent
- **Eventual consistency** – data becomes consistent over time

► Why BASE is used in NoSQL

- Designed for distributed, large-scale environments
- Prioritises performance and scalability over strict consistency
- Suitable for real-time web, mobile, IoT applications



► **ACID in RDBMS (PostgreSQL / Oracle)**

- Enforces correct, predictable transactions under a strict schema
- Ensures durability through WAL (Write-Ahead Logging) and checkpointing
- Suitable for banking, ERP, inventory, and mission-critical records

► **BASE in NoSQL (MongoDB Documentation)**

- Prioritises throughput and latency over immediate consistency
- Replica sets sync asynchronously for speed
- Enables flexible schema evolution without table redesign



Databases and Applications

CAP Theorem – Distributed Systems

► CAP Theorem States:

- A distributed system can guarantee only **two** of:
- Consistency (C)
- Availability (A)
- Partition Tolerance (P)

► Real-world Interpretation

- Network failures force trade-offs between C and A
- No database can provide all three at all times
- NoSQL DBs balance A+P; RDBMS balances C+P



► The CAP Trade-off (Academic View)

- **Consistency (C):** Every read returns the most recent committed write
- **Availability (A):** Every request receives a non-error response
- **Partition Tolerance (P):** System continues despite message loss or network splits

► Implications

- In a real distributed system, network partitions are unavoidable
- Systems must choose **CA**, **AP**, or **CP** depending on business need
- No system can simultaneously guarantee C, A, and P in the presence of failures



Databases and Applications

RDBMS vs NoSQL – When to Use What?



► RDBMS (MySQL, PostgreSQL)

- Structured schema with fixed tables
- Strong ACID guarantees
- Best for finance, transactions, ERP systems
- Fixed schema with strong typing and relational integrity constraints
- Vertical scaling (scale-up) using single-node performance
- Optimised for OLTP workloads and structured datasets

► NoSQL (MongoDB, Cassandra)

- Flexible schema for semi-structured data
- Scales horizontally across clusters
- Schema-flexible documents enabling heterogeneous records
- Horizontal scaling (scale-out) through sharding and replica sets

Databases and Applications

Unit III

MongoDB CRUD: insert, update, delete

Pooja T S

Computer Applications



Databases and Applications

Introduction to MongoDB

► What is MongoDB?

- A NoSQL, document-oriented database storing data as BSON (Binary JSON).
- Uses flexible, schema-less collections instead of rigid tables.
- Designed for scalability, high availability, and handling semi-structured data.

► Where MongoDB is used

- Web apps, analytics, IoT data, product catalogs, student records, logs.
- Best suited where structure evolves frequently.



Databases and Applications

MongoDB vs SQL – Quick Comparison

► Key Concepts

- SQL: tables, rows, fixed schemas.
- MongoDB: collections, documents, flexible schemas.

► Why choose MongoDB?

- Easy to store nested fields (arrays, objects).
- No schema migrations required.
- Faster prototyping for modern applications.



Databases and Applications

Mongo Shell – Getting Started

► What is the Mongo Shell?

- An interactive command-line interface to communicate with MongoDB.
- Supports JavaScript-like command syntax.
- Used for administration, querying, and CRUD operations.

► Basic Shell Commands

- `mongo` — start shell.
- `show dbs` — list databases.
- `use college` — switch/create database.
- `show collections` — list collections.



Databases and Applications

MongoDB Data Types – Overview

- ▶ **MongoDB supports flexible data types:**
 - **String** – Most common text data type. ("ABC")
 - **Number** – int, long, double. (21, 45.6)
 - **Boolean** – true / false.
- ▶ Used in documents to represent text, numeric values, flags, and simple state information.



► Advanced supported types:

- **Array** – Stores multiple values. (`["Python", "SQL"]`)
- **Object / Embedded Document** – Nested fields. (`{ city: "Bengaluru" }`)
- **ObjectId** – Auto-generated unique identifier.

- ### ► Supports flexible, schema-less structure with nested and repeated fields.



► Other important types:

- **Date** – Stores timestamps. (`ISODate(...)`)
 - **Null** – Missing / unknown value.
 - **Binary Data** – Images, PDFs, encoded content.
- These types help store structured, semi-structured, and application-specific data.



- ▶ **MongoDB stores data as BSON (Binary JSON).**
 - Accepts standard JSON input in the shell.
 - Internally converts JSON to efficient BSON.
 - Supports nested fields and arrays naturally.
- ▶ **A document is a collection of key-value pairs.**
 - Keys are strings; values may be any valid datatype.
 - Fields may contain nested documents.
 - Order of fields is preserved but not required.



► Document = key-value structure

- Uses JSON-like syntax stored as BSON.
- Fields may be strings, numbers, arrays, or embedded docs.
- No fixed schema — fields can differ across documents.

► Example student document

```
{ name: "Aarathi", age: 21, dept: "CSE" }
```



Databases and Applications

Sample Document – Good Structure

► Document with nested fields:

```
{  
  name: "Riya",  
  age: 20,  
  skills: ["Python", "MongoDB"],  
  address: { city: "Bengaluru", pincode: 560050  
}
```

► Why this structure works well:

- Natural representation of real-world objects.
- Easy to extend with new fields.
- Supports deep queries using dot notation.



Databases and Applications

Rules for Writing MongoDB Queries

► Important rules:

- Keys must always be in double quotes when nested.
- Queries are always written inside { }.
- Multiple conditions use \$and, \$or, or implicit AND.

► Examples

```
db.students.find({ age: 21, dept: "CSE" })  
db.students.find({ $or: [ {dept:"CSE"}, {age:22}  
] })
```



Databases and Applications

CRUD — Insert Operations

► Insert a single document

```
db.students.insertOne({ name: "Amit", age: 21,  
dept: "CSE" })
```

► Insert multiple documents

```
db.students.insertMany([ { name: "Riya", age:  
20, skills: ["Python"] }, { name: "Kiran", age:  
22, skills: ["SQL","Java"] } ])
```

► Where inserts are used

- Adding new student profiles.
- Importing bulk data from apps/forms.



Databases and Applications

CRUD — Read (Find) Operations

► Read all documents

```
db.students.find()
```

► Find with filters

```
db.students.find({ dept: "CSE" })
```

```
db.students.find({ age: { $gt: 20 } })
```

► Projection (show selected fields)

```
db.students.find({}, { name: 1, dept: 1 })
```



Databases and Applications

CRUD — Update Operations

► Update one document

```
db.students.updateOne( { name: "Amit" }, { $set:  
  { age: 22 } } )
```

► Update many documents

```
db.students.updateMany( { dept: "CSE" }, { $inc:  
  { age: 1 } } )
```

► Why updates are essential

- Correcting records, app-based profile updates.



Databases and Applications

CRUD — Delete Operations

► Delete a single document

```
db.students.deleteOne({ name: "Kiran" })
```

► Delete multiple documents

```
db.students.deleteMany({ dept: "CSE" })
```

► Important Note

- Deletions are permanent — use filters carefully.
- Ideal for removing inactive user profiles.

Databases and Applications

Unit III

MongoDB Query Filters and Operators

Pooja T S

Computer Applications



Databases and Applications

MongoDB Query Operators – Comparison

► Comparison Operators

- \$eq – equals
- \$gt, \$lt – greater/less than
- \$gte, \$lte – greater/less than or equal

► Examples

```
db.students.find( { age: { $gt: 20 } } )
```

```
db.students.find( { dept: { $eq: "CSE" } } )
```



► Frequent beginner errors

- Writing `age > 20` instead of using `$gt`.
- Missing braces: must use `{ age: { $gt: 20 } }`.
- Forgetting that string comparison matches exactly.

► Correct pattern

```
db.students.find({ age: { $lte: 22 } })
```



Databases and Applications

MongoDB Logical Operators

► Logical Operators

- \$and – combine multiple conditions
- \$or – match at least one condition
- \$not, \$nor – negation patterns

► Examples

```
db.students.find({ $and:  [{dept:"CSE"}, {age:{  
$gt:20 }}] })
```

```
db.students.find({ $or:  [{age:21}, {dept:"ECE"}]  
})
```



Databases and Applications

Logical Operators – Usage Guide

► When to use

- Use \$and when filtering on 2+ conditions.
- Use \$or to search alternatives.
- Use \$not only inside another condition.

► Common mistakes

- Writing {age:21, \$or:[...]} (where ordering fails).
- Forgetting arrays inside \$and and \$or.
- Using SQL syntax: dept="CSE" AND age>20.



Databases and Applications

MongoDB Array Operators

► Array Operators

- `$in` - match any value from a list
- `$all` - must contain all listed values
- `$size` - array length equals

► Examples

```
db.students.find({ skills: { $in: ["Python","SQL"]  
} })
```

```
db.students.find({ skills: {  
$all: ["Python","MongoDB"] } })
```



Databases and Applications

Array Queries – Mistakes to Avoid

► Common mistakes

- Expecting \$in to behave like AND.
- Using "skills.python" instead of array filters.
- Misusing \$size expecting ranges.

► Correct usage

```
db.students.find({ skills: { $size: 2 } })
```

Use \$all for “must contain both”.



Databases and Applications

Element Operators – exists, type

► Element Operators

- \$exists – checks if a field is present.
- \$type – matches data type.

► Examples

```
db.students.find({ address: { $exists:true } })  
db.students.find({ age: { $type:"int" } })
```



Databases and Applications

Querying Nested Fields

► Using dot notation

```
db.students.find({ "address.city": "Bengaluru" })
```

► When to use

- For structured data such as addresses, contacts, marks.
- Easier than joins in SQL.

► Common mistakes

- Using single quotes inside shell.
- Typing `address.city = "Bengaluru"` (SQL syntax).



PES
UNIVERSITY

CELEBRATING 50 YEARS

Pooja T S
Assistant Professor
Department of Computer Applications
poojats@pes.edu
080-26721983 Extn: 391



Databases and Applications

Unit III

Pooja T S

Computer Applications

Databases and Applications

Unit III

Schema Design in MongoDB: Embedded vs Referenced

Pooja T S

Computer Applications



► Schema design is critical even in NoSQL

- MongoDB allows flexible document structures
 - No fixed schema, but structure affects performance
 - Poor design increases query cost
 - Data duplication might grow uncontrollably
- Designing based on *query patterns*
 - Embed when data is always read together
 - Reference when relationships grow large
 - Reduce round-trips for frequently accessed data



Databases and Applications

Schema Design in MongoDB – Overview



- ▶ MongoDB stores data as **documents** in **collections**, not rows in tables.
 - Documents are JSON-like structures (stored internally as BSON)
 - A collection can hold documents with flexible structure
- ▶ Schema design in MongoDB focuses on **how documents relate** to each other:
 - **Embedded** design – store related data inside a single document
 - **Referenced** design – store related data in separate collections and link them



Databases and Applications

Embedded Documents – Concept

- ▶ In an **embedded schema**, related data is stored **inside** the same document.
 - One document contains all information needed for a use-case
 - No separate lookup is required in another collection
- ▶ Good when:
 - Relationship is **one-to-few** (limited number of related items)
 - Related data is **always accessed together** (high read locality)
 - Duplication is acceptable and does not grow unbounded



Databases and Applications

When to Use Embedded Schema

► Choose embedding when:

- Data is accessed together 90% of the time
 - Example: blog post + comments
 - Example: invoice + line items
 - Example: student + list of enrolled courses
- The related data has limited or predictable growth
 - Customer has 2-5 addresses only
 - Product has fixed set of specifications
 - Order rarely exceeds 10-20 items



Databases and Applications

Embedded Documents – Example

► Example: Order document with embedded customer

```
{  
  _id: ObjectId("..."),  
  orderId: "0101",  
  orderDate: ISODate("2025-01-10"),  
  totalAmount: 2499,  
  customer: {  
    customerId: "C001",  
    name: "Amit Kumar",  
    phone: "98765...",  
    city: "Bengaluru"  
  }  
}
```



Databases and Applications

Referenced Documents – Concept

- ▶ In a **referenced schema**, related data is stored in **separate documents**.
 - One document holds the main entity
 - Another document holds related entity, linked by an identifier
- ▶ Good when:
 - Relationship is **one-to-many** or **many-to-many**
 - Related data is reused across multiple documents
 - You want to avoid large document growth or duplication



Databases and Applications

When to Use Referenced Schema

► Choose referencing when:

- Data grows without bound
 - Customer may place 500+ orders
 - Product may have thousands of reviews
 - A category links to thousands of SKUs
- Data is reused across many documents
 - City list, department list, categories
 - Reduces duplication of standard information
 - Updates become easier and more consistent



Databases and Applications

Referenced Documents – Example

► Customers collection:

```
{  
  _id: ObjectId("..."),  
  customerId: "C001",  
  name: "Amit Kumar",  
  phone: "98765...",  
  city: "Bengaluru"  
}
```

► Orders collection:

```
{  
  _id: ObjectId("..."),  
  orderId: "0101",  
  orderDate: ISODate("2025-01-10"),  
  totalAmount: 2499,  
  customerId: "C001"
```

}



► Embedded – Strengths

- Fastest reads (single document)
 - Optimised for read-heavy workloads
 - Minimises number of queries
 - Ideal for hierarchical data
- Simple structure for client applications
 - Easier to serialize/deserialize
 - All data stays in one JSON object
 - No need for additional API calls



► **Embedded** design:

- Store related information **inside** one document
- Fewer queries, excellent read performance for that document
- Can lead to duplication if same embedded data repeats many times

► **Referenced** design:

- Store related information in **separate** collections
- Requires joining in application code (multiple queries)
- Avoids duplication, suitable for shared data (e.g., same customer in many orders)



► Rule 1: Model data based on application queries

- Schema follows the read patterns
 - Optimise for most frequent queries
 - Minimise number of lookups
 - Reduce joins at application layer

► Rule 2: Consider update frequency

- Frequently updated fields → use reference
- Rarely updated fields → embed
- High-write workloads prefer smaller docs



► Rule 3: Think about document size

- MongoDB document limit is 16 MB
 - Large arrays → move to separate collection
 - Big logs/history → reference
 - Avoid uncontrolled growth within documents

► Rule 4: Think about cardinality

- Low cardinality → embed
- High cardinality → reference
- Many-to-many → always reference



- ▶ **Single collection:** orders with embedded customer

Collection: orders

```
_id: ObjectId("...")
orderId: "0101"
orderDate: ISODate("2025-01-10")
totalAmount: 2499
customer: {
  customerId: "C001"
  name: "Amit Kumar"
  phone: "98765..."
}
```

- ▶ **Interpretation:**

- All order + customer details are stored in a **single document**
- Best for cases where a customer has a small to moderate number of orders



- ▶ **Two collections:** customers and orders

Collection: customers

```
_id: ObjectId("...")  
customerId: "C001"  
name: "Amit Kumar"  
phone: "98765..."  
city: "Bengaluru"
```

Collection: orders

```
_id: ObjectId("...")  
orderId: "0101"  
orderDate: ISO("2025-01-10")  
totalAmount: 2499  
customerId: "C001"
```

customerId reference: "C001" → links customers → orders



► **Prefer Embedded** when:

- You read the parent and child data **together** most of the time
- Child collection size is small (one-to-few)
- Data duplication does not create maintenance issues

► **Prefer Referenced** when:

- A child (e.g., customer) is shared across many parents (many orders)
- You want a single, central source of truth for the entity
- Documents would otherwise become very large or frequently updated

Databases and Applications

Unit III

Indexing: Single, Compound, Text in MongoDB

Pooja T S

Computer Applications



Databases and Applications

MongoDB Indexing – Introduction

► What is an Index?

A data structure that stores keys in a **sorted** manner.

Helps MongoDB locate documents **without scanning entire collections**.

Based on a **B-tree model**, enabling fast lookups and range queries.

► Why Indexes Matter?

Improve read performance significantly.

Reduce query latency for filters and sorting.

Essential for large datasets and real-time applications.



Databases and Applications

How MongoDB Uses Indexes?

► Execution Flow

Query Planner checks available indexes.

Chooses the **best index** using cost-based optimisation.

Avoids collection scan when a matching index exists.

► Index Structure

Uses a **B-tree** to store ordered key-value references.

Supports equality, range, prefix and sort queries.

Allows compound indexes to cover multiple fields.



Databases and Applications

Creating Indexes

► Single-Field Index

Syntax: `db.collection.createIndex({ field: 1 })`

1 = ascending, -1 = descending

Improves queries like: `find({ field: value })`

► Compound Index

Created as: `db.collection.createIndex({ a:1, b:1 })`

Order matters for query planning.

Supports prefix-based lookups.



Databases and Applications

Index Types in MongoDB

▶ Common Index Types

Single Field Index – best for equality lookups.

Compound Index – multi-column search.

Multikey Index – for array fields.

▶ Specialised Index Types

Text Index – for search on textual content.

Hashed Index – uniform hashing for sharding.

Geospatial Index – for location-based queries.



Databases and Applications

Index Properties in MongoDB

► Unique Indexes

Enforces uniqueness of field values.

Syntax: `createIndex({email:1}, {unique:true})`

Used for usernames, emails, IDs.

► Sparse + TTL Indexes

Sparse: Includes only documents with indexed field.

TTL: Auto-deletes documents after given seconds.

Syntax: `expireAfterSeconds:3600`



Databases and Applications

Index Builds in MongoDB

► **Foreground Build**

Blocks writes but is fastest.

Suitable for offline maintenance.

Not recommended for production workloads.

► **Background Build**

Allows reads/writes during build.

Slightly slower but safer in production.

Use: `{ background: true }`



Databases and Applications

Managing Indexes

► View Existing Indexes

```
db.collection.getIndexes()
```

Shows fields, properties, and index name.

Helps verify accidental large indexes.

► Dropping Indexes

Drop one: `db.collection.dropIndex("name")`

Drop all: `db.collection.dropIndexes()`

Avoid excessive index count to save memory.



Databases and Applications

Measuring Index Performance

► Using explain()

`explain("executionStats")` reveals plan used.
Checks if index prevents COLLSCAN.
Helps diagnose slow queries.

► Key Metrics

Index hit ratio.
Keys examined vs documents examined.
Query execution time (ms).



Databases and Applications

Indexing Strategy – Best Practices

► What to Index

Fields used in frequent queries.

Fields used in sorting operations.

Foreign-key-like fields (customerId etc.).

► What Not to Index

High-cardinality fields rarely queried.

Booleans (true/false) – low selectivity.

Large multi-key arrays – expensive to maintain.



Databases and Applications

Text Indexes – Concept

- ▶ What is a text index?
 - Special index type for full-text search on string content
 - Tokenises text into words and indexes them
 - Used with `$text` queries
- ▶ Rules and limits
 - At most **one** text index per collection
 - Can include multiple string fields in the same text index
 - Does not support partial word match (no prefix search like `"dev*"`)



Databases and Applications

Text Indexes – Example

- ▶ Blog posts collection: posts
 - Fields: title, body, tags
 - Need to search posts by keywords in title or body
 - Use a combined text index
- ▶ Shell commands
 - `db.posts.createIndex({ title:"text", body:"text" })`
 - Search: `db.posts.find({ $text:{ $search:"mongodb index" } })`
 - Relevance score: `project { score:{ $meta:"textScore" } }`



Databases and Applications

Indexing – Putting It All Together

► Choosing index types

- Single field: simple filters or sort on one field
- Compound: frequent multi-field filters and sorts (respect prefix rule)
- Text: keyword search on large string fields (`$text` queries)

► Good practices

- Index only fields that appear in `find()`, `sort()`, or join-like patterns
- Avoid too many indexes on write-heavy collections
- Use `db.collection.explain()` to confirm index usage